Supplemental Document for: Visual Fingerprints of Vibration Signals Using Time Delay Embeddings

J. Rakuschek[1 [0009−0000−9387−1785]], A. Boesze[2], J. Schmidt[3 [0000−0002−9638−6344]], and T. Schreck[1 [0000−0003−0778−8665]]

[1] TU Graz, Institute of Visual Computing, Austria
[2] Binder+Co AG, Austria
[3] TU Wien, Institute of Visual Computing & Human-Centered Technology, Austria

1 Time Delay Embedding: Definition and Parameters

The Time Delay Embedding (TDE) originates from the field of *Topological Data Analysis* (TDA), with *Takens' Embedding Theorem* as its underlying theoretical foundation [2]. Given a time series as a function $f(t)$ depending on a time variable t, the TDE extracts a sliding window view, resulting in a mapping function TDE [1]:

$$\mathrm{TDE}_{d,\tau} f : \mathbb{R} \to \mathbb{R}^d, t \to \begin{pmatrix} f(t) \\ f(t+\tau) \\ f(t+2\tau) \\ \vdots \\ f(t+(d-1)\tau) \end{pmatrix}$$

In definition (1), d refers to the dimension of the TDE, that is, the window size in the sliding window view, while τ refers to the delay within the window. Additionally, a stride s is defined as the spacing between windows. In this work, we consistently set τ and s both to 1. When setting the parameters to greater values, the point cloud becomes thinner while the resulting circular patterns remain. Table 1 provides an example to illustrate this effect with varying parameters. While the parameters τ and s are central for the time delay embedding in other domains, we observed that it does not provide beneficial effects for values greater than 1 in the case of vibrations. We provide an example implementation in Python for the proposed fingerprint in Section 3.

2 TDE With Increasing Noise Level

In this section, we provide further details on the experiment shown in Figure 3 in the original paper. In this experiment, we iteratively add noise to a vibration signal and examine the corresponding TDE and the spectrogram at each iteration step. Let S_0 denote the original signal. At each iteration i, we add noise to the signal. The noise added at each step is given by $\mathrm{Noise}_i = 0.5 \times N_i$, where

N_i is a vector of random values drawn from a standard normal distribution, $N_i \sim \mathcal{N}(0, 1)$. The scaling factor 0.5 is chosen to control the added noise level, such that the experiment can be illustrated in multiple iteration steps with smaller changes inbetween steps. The signal after the i-th iteration, S_i, can be expressed recursively as:

$$S_i = S_{i-1} + \text{Noise}_i$$

By substituting the expression for Noise_i, we get:

$$S_i = S_{i-1} + 0.5 \times N_i$$

Expanding this recursively, the signal after i iterations can be expressed in terms of the original signal and the cumulative noise:

$$S_i = S_0 + \sum_{j=1}^{i} 0.5 \times N_j$$

This formulation allows us to analyze the signal at each step of the iteration process, the result is presented in Figure 1 in this document and is meant as an expansion with more details to Figure 3 shown in the original paper.

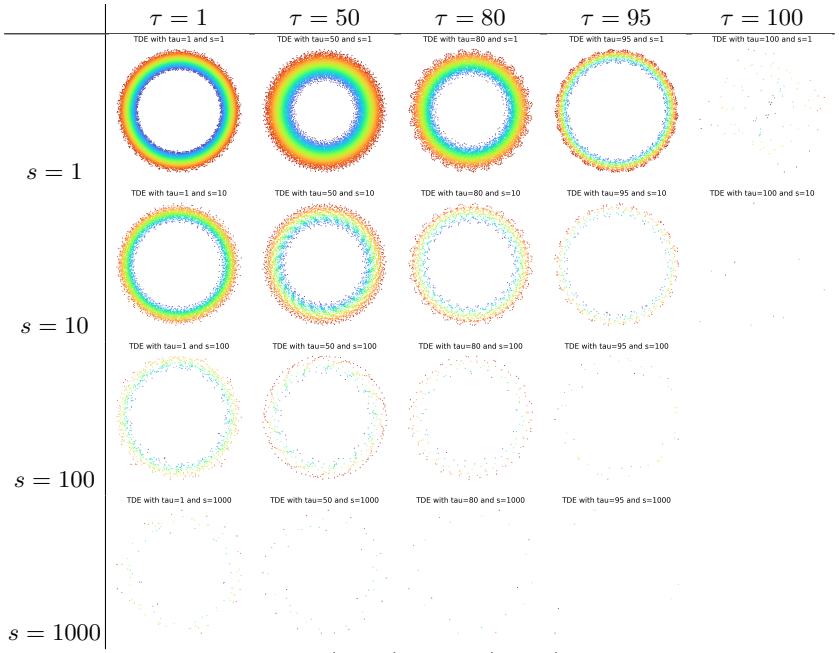

Table 1. Effect of parameters τ (delay) and s (stride) on the TDE, it can be seen that larger values for τ and s result in a thinner point cloud while the overall circular structure remains. The TDE has been generated from vibrations obtained from a hydro power plant with 100.000 data points within the time series.

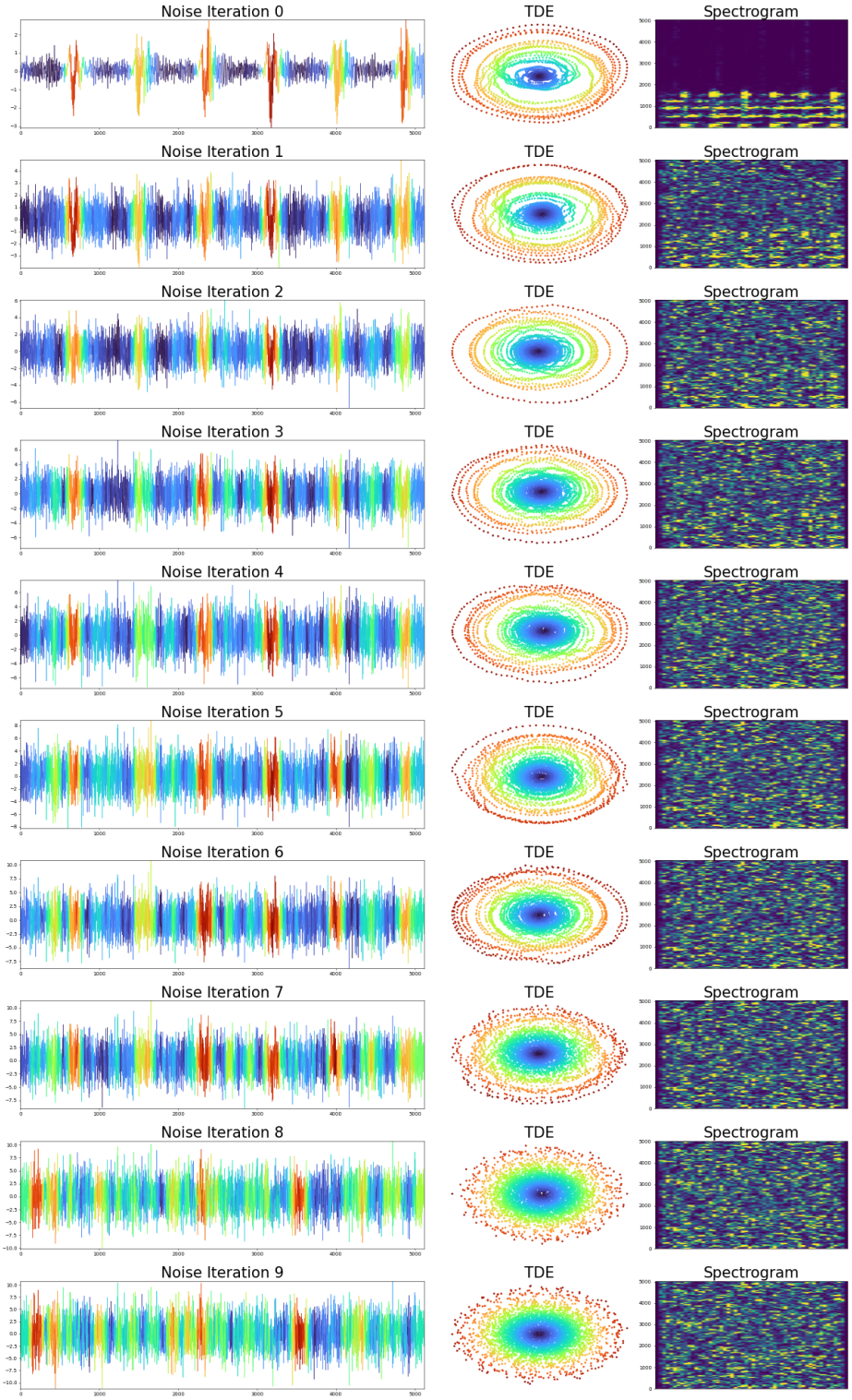

Fig. 1. The vibration is polluted with increasing noise level in each iteration.

3 Appendix A: Python Code

The subsequent script demonstrates the fingerprint visualization through a Python implementation.

```python
# Required Python libraries:
# - numpy
# - matplotlib
# - scikit-learn
# Install via: pip3 install numpy matplotlib scikit-learn

import numpy as np
from matplotlib.pyplot import colormaps
from matplotlib import pyplot as plt
from sklearn.decomposition import PCA

def time_delay_embedding(values, d, tau=1, stride=1):
    values = np.asarray(values)
    windows = []
    index = 0
    while index <= len(values) - d:
        window = []
        for i in range(index, index + d * tau, tau):
            if i >= len(values):
                return np.array(windows)
            window.append(values[i])
        windows.append(window)
        index += stride
    return np.array(windows)


def plot_embedding(ax, tde, title="TDE"):
    projected = PCA(n_components=2).fit_transform(tde)
    scores = []
    for point in projected:
        scores.append(np.linalg.norm(point))
    scores = np.array(scores)
    scores_norm = (scores - np.min(scores)) / (np.max(scores) - np.min(scores))
    ax.scatter(projected[:, 0], projected[:, 1], s=8, c=colormaps["turbo"](scores_norm))
    ax.axis("off")
    ax.set_title(title, fontsize=30)

# Input: values.npy as a 1D numpy array, ideally a vibration
values = np.load("values.npy")
fig, ax = plt.subplots(nrows=1, ncols=1)
tde = time_delay_embedding(values, d=100, tau=1, stride=1)
plot_embedding(ax, tde)
plt.savefig("tde.png")
```

References

1. Perea, J.A.: Topological Time Series Analysis. ArXiv **abs/1812.05143** (2018). https://doi.org/10.1090/NOTI1869
2. Takens, F.: Detecting strange attractors in turbulence (1981). https://doi.org/10.1007/BFB0091924